

Code-Layout, Readability and Reusability

Date	7 July 2026
Team ID	XXXXXX
Project Name	EduGenie Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout The code	YES	Follows standard PEP 8 spacing rules.
2	Proper File Structure	Files and folders are logically organize	YES	Separates FastAPI routers, templates, and static assets
3	Meaningful Variable Names	Variables reflect their purpose clearly	YES	Variables like gemini_api_key and quiz_topic are explicit.
4	Function / Method Names	Functions are descriptively named	YES	Uses clean action verbs like generate_quiz and get_summary.
5	Code Comments	Inline and block comments explain logic	YES	Critical AI inference steps are well-documented.
6	Modular Design	Code is split into reusable functions/modules	YES	AI core operations are completely isolated from routing logic.
7	No Redundant Code	Duplicate or unused code is removed	YES	Shared prompt structures are centralized across modules.
8	Error Handling	Exceptions and errors are handled gracefully	YES	Includes comprehensive HTTP try/except checks for API timeouts.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Database Connection Manager (database.py)	Python / SQLAlchemy	Reused across all service endpoints requiring persistent storage operations, including user	High

			registration validation, quiz metrics tracking, and roadmap storage.	
2	Gemini API Inference Handler (gemini_service.py)	Python / Google GenAI SDK	Centralized wrapper utility reused across the Intelligent Q&A Module and the Learning Path Recommendation Engine to pass structured prompt payloads.	High
3	Base Navigation Master Layout (base.html)	HTML5 / CSS3	Extensively reused as the master template shell across all specific feature views including the Home, Quiz, and Summarization dashboard interfaces via Jinja2 template inheritance.	High
4	Response Structure Formatter Utility (utils.py)	Python / Regular Expressions	Shared across the Quiz Generation pipeline and Text Summarization module to sanitize and cleanly parse raw markdown strings into structured UI layouts.	Medium
5	Local NLP Tokenizer Setup (local_model.py)	Python / Transformers	Reused within local preprocessing workflows to handle content size validation strings before executing heavy inference loops.	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	The application perfectly follows PEP 8 standards with uniform indentation. Directories logically isolate routers, service components, local transformers scripts, and server-side template bundles.
Readability	5	Explanatory naming choices like gemini_api_key and generate_quiz eliminate guesswork. Complex asynchronous API streams and local inference token paths are clean and direct.
Reusability	4	Shared base layouts, centralized third-party API handlers, and unified database connection singletons are heavily cross-leveraged, drastically dropping codebase line counts.
Documentation / Comments	5	All multi-step prompt engineering frameworks, internal model setups, and custom regex cleanup routines contain clear block annotations explaining behavioral reasoning.
Overall Score	4.75	The implementation represents a high-grade, production-ready system architecture profile. It achieves excellent clean code characteristics while gracefully bridging local and cloud dependencies.